

Incremental Parsing for Structured Editors

Mira Castell

Ravi Anand

Abstract

A structured editor reparses the whole file after every keystroke. We reparse only the smallest subtree an edit can have changed, and show that the trees we keep are the trees a full parse would build.

1 Introduction

Editors that understand the code they hold parse it again after every change, and on a large file the parse takes longer than the gap between two keystrokes. Most edits change one token, and one token can change only the subtrees that contain it. We find the smallest such subtree, parse it again, and splice the result back into the tree. This paragraph runs on for several lines so that the column has a measure of its body text, as a real paper’s column would, and the reader can tell a full line from a short one.

The incremental parser is never slower than a full parse, and on files of ten thousand lines it is a hundred times faster.

2 Method

The parser keeps, for every node, the range of tokens it covers and the state the parser was in when it began the node. An edit is mapped to the tokens it touches, and the search for a subtree to reparse starts there.

1. Smallest enclosing node. Walk up from the edited token until the node’s range covers the whole edit and its starting state is one the edit cannot have changed, then reparse that node alone.

2. Splicing. Replace the old node with the new one and shift the ranges of every node after it by the length the edit added:

```
node = enclosing(tree, edit)
fresh = parse(text, node.start)
tree.replace(node, fresh)
for later in tree.after(node):
    later.shift(edit.delta)
```

Algorithm 1 Incremental reparse.

```
1: input tree  $T$ , edit  $e$ 
2:  $n \leftarrow \text{TOKEN}(T, e)$ 
3: while  $\neg \text{COVERS}(n, e)$  do
4:    $n \leftarrow \text{PARENT}(n)$ 
5: end while
6: return  $\text{SPLICE}(T, n)$ 
```

Algorithm 1 is all the parser does on an edit; the rest of its time is spent waiting for the next one.

3 Checking the Trees

After every edit in the test corpus we parse the file again in full, and compare the two trees node by node. They have never differed. The check runs in the background and costs the user nothing.

```
input: trees  $A, B$ 
1: if  $\text{KIND}(A) \neq \text{KIND}(B)$  then return false
2: for each child pair  $(a, b)$  do
3:   if not  $\text{SAME}(a, b)$  then return false
4: return true
```

Algorithm 2: Comparing two trees.

Algorithm 2 compares two trees from the root down, and stops at the first pair of nodes that differ.

```
1:  $q \leftarrow [T]$ 
2: while  $q$  is not empty do
3:    $n \leftarrow \text{POP}(q); \text{CHECK}(n)$ 
4:   push the children of  $n$  on  $q$ 
5: end while
```

Algorithm 3: Visiting every node.

Algorithm 3 visits the nodes breadth first, so a difference near the root is found before one deep in the tree.

4 Conclusion

Reparasing the smallest subtree an edit can change keeps a structured editor responsive on files of any size, and costs nothing in correctness.